

Canabalt

A didactic C++ port of the one-button runner — build, play & mechanics

Built on the Otacon engine (see `docs/otacon-engine.pdf`)

Contents

1	About Canabalt	2
1.1	Faithfulness	2
2	Building and running	3
2.1	macOS / Linux	3
2.2	Windows	3
2.3	Vulkan note	3
3	How to play	4
3.1	Controls	4
4	The incremental modes	5
5	Game mechanics	6
5.1	Player physics (from <code>Player.m</code>)	6
5.2	The level generator (<code>Sequence.cpp</code>)	6
5.3	Background & foreground extras	7
6	Presentation	8
6.1	Progressive sprite reveal (F6 / F8)	8
6.2	Sound & music	8
7	Assets	9
8	Teaching toggles (quick reference)	10

Chapter 1

About Canabalt

Canabalt is the original one-button infinite runner: a lone figure sprints across the rooftops of a collapsing city, and the only control is **jump**. Hold longer to jump higher; clear the gaps; survive as far as you can. There is no winning — only your distance in metres, and your best.

This is a teaching-oriented C++ port. The source of truth is the official open-source release ([ericjohnson/canabalt-ios](#)), an Objective-C game on the `flixel-ios` framework, kept under [reference/](#) and studied closely so the port does **not diverge** from the original’s feel. The game is written entirely against the Otacon engine through its `IGame` interface; everything game-specific — code, assets and these docs — lives under `src/game/canabalt/`.

1.1 Faithfulness

The simulation reproduces flixel’s exact integrator, velocity model, swept-AABB collision and follow camera, plus Canabalt’s exact player physics constants (Chapter 5). With the default float scalar the run is deterministic and reproducible; the same seed yields the same city every time.

Chapter 2

Building and running

Canabalt is the `canabalt` executable; it links the Otacon static library.

2.1 macOS / Linux

```
cmake -S . -B build          # first configure fetches the window library
cmake --build build -j8
./build/bin/canabalt
```

2.2 Windows

Visual Studio: open the folder (it has `CMakeLists.txt`); VS configures automatically. Pick the `canabalt.exe` target and Run.

VS Code: with the *CMake Tools* extension, select a kit, then Build and Run. Or from a terminal:
`cmake -S . -B build && cmake --build build.`

2.3 Vulkan note

If built with the Vulkan backend on macOS, launch through `tools/run.sh` (it sets the MoltenVK loader environment). Any backend plays identically.

Chapter 3

How to play

You start on the **title screen**; press **jump** to begin. From then on you run automatically and ever faster — tap jump to hop a gap, hold it for a higher leap. Falling into a gap, smacking a wall, or being caught by a hazard ends the run.

- **Distance.** The metres-run counter sits top-right.
- **Game over.** On death a panel shows how far you ran and how you died (“...before falling to your death”, “...smashing into a wall”, “...turning into a fine mist”). Press **jump** or **R** to run again, or click **EXIT** (top-right) to return to the title.
- **High score.** Beat your best and a **NEW RECORD** badge appears; the best distance is saved to `src/game/canabalt/assets/highscore.dat` and persists between sessions.

3.1 Controls

Key / input	Action
Space / W / Up / Left-click	Jump (hold for a higher jump); start; retry
R	Retry the current run
Click EXIT (on game over)	Back to the title
1 ...4	Jump directly to that mode
] or . / [or ,	Next / previous mode
F9	Mute / unmute all audio
F10	Cycle the music track (Run / Daring Escape / Mach Runner)
M	Show / hide the mouse cursor
` (backtick)	Hide / show ALL dev UI — clean play
Esc	Quit

The teaching toggles (F1–F8, H, +/-, C, G, ...) are documented with the modes below and in the engine manual; the bottom legend always shows the live state. Press ` any time to hide all of it and just play.

Chapter 4

The incremental modes

The build is staged as four runnable “modes” that show the game assembled from a single box up to the full runner. Cycle with `]`/`[`, or jump straight to one with the number keys 1–4.

1. **Static Box** — a box on the ground. Geometry + camera only.
2. **Gravity** — a box falls and lands on a platform (the integrator and collision). Press `Space` to drop again; press `E` to burst particles at the cursor.
3. **Run & Jump** — the real `Player` auto-runs on flat ground; hold to jump; the camera follows. Full original feel, no hazards.
4. **Infinite Run** — the real game: the ported level generator, the parallax city, hazards, sprites, sound and the title/score flow.

Chapter 5

Game mechanics

5.1 Player physics (from `Player.m`)

Quantity	Value
Hitbox	16×18 (sprite 30×30 , offset 6, 12)
Start velocity x	125
Gravity (accel. y)	1200
Drag x	640
Max velocity	(1000, 360)
Accel. x ramp	60/36/24/12/4 as speed rises (fast then tapering)
Jump limit	$v_x / (1000 \cdot 2.5)$, capped at 0.35 s
Short / full jump v_y	$-0.65 \cdot 360$ / -360
Fall death	$y > 484$
Wall death	accel. $x \rightarrow 0$, $v_x \rightarrow 0$

The hold-to-jump window scales with speed: the faster you run, the higher you can leap. A hard landing makes the runner stumble and roll.

5.2 The level generator (`Sequence.cpp`)

A faithful port of the original `Sequence.m`. Exactly two “sequences” leapfrog: each owns one wide building, and when one scrolls off the left it regenerates just past the other. Each new building’s gap, minimum width and vertical drop are derived from your current `velocity.x` and `jumpLimit` — so the level gets harder precisely as you get faster, exactly like the original. The first building is a fixed launch hallway.

Special building archetypes are generated by type, each with its own collision shape, art and behaviour:

- **Hallway** — a tunnel you run through; its windows smash (glass + sound) as you pass, and doors line the back wall (the launch tunnel has none).
- **Collapse** — the roof sinks the moment you step on it; you ride it down and must leap off before it drops away.
- **Bomb** — a bomb whistles down and detonates on the roof; touching it is fatal.
- **Crane** — a metal beam you cross like a platform (footsteps ring out as metal).

- **Billboard** — a thin catwalk ledge under a damaged sign.
- **Leg** — a giant mech leg stomps down and crushes the building below.

Rooftops also scatter obstacles (crates you stumble over) and pigeons that flush as you approach. Roof and floor caps, wall and window styles all vary per building.

5.3 Background & foreground extras

Distant Easter eggs ride the skyline — a mothership, a dark tower, two marching mech *walkers* (which vent smoke), and a jet that periodically streaks past and rattles the screen — while a steel girder occasionally sweeps the foreground. A startup screen-quake (the collapsing city) shakes the camera as a run begins.

Chapter 6

Presentation

6.1 Progressive sprite reveal (F6 / F8)

The world can be peeled between pure collision geometry and full sprites one layer at a time. **F6** cycles the level — **geometry** (everything is boxes) → **player** (the runner becomes its **player2.png** sprite) → **buildings** (rooftops tile a wall texture) → **background** (parallax skyline) → **full sprites** (textured particles) — and **F8** jumps straight between all-geometry and all-sprites. **C** overlays the collision boxes at any level, so you can see the geometry under the art.

6.2 Sound & music

Sound effects (footsteps, jumps, window smashes, glass, the bomb/leg/collapse hazards, the jet, pigeon flaps, the death thud) are decoded in-house from the original **.caf** files and mixed live. Three looping music tracks are included; **F9** mutes all audio and **F10** cycles the track. The title screen has its own theme.

Chapter 7

Assets

All game assets live under `src/game/canabalt/assets/` and are loaded through the engine's in-house decoders — nothing is baked into the binary:

- `images/` — PNGs (player sheet, HUD glyphs, building/crane/billboard tiles, UI: title, game-over, paused, cursor). Decoded by the in-house PNG decoder; `images/raw/` holds the un-atlased source art.
- `sound/` — sound effects as 22050 Hz mono 16-bit LPCM `.caf`. `convert.sh` regenerates them from the `.mp3` sources via `afconvert`.
- `music/` — the looping tracks, converted to `.caf` the same way.

Chapter 8

Teaching toggles (quick reference)

These flip engine/game behaviour live so the effect is visible at once; the HUD's second line shows the current state.

Key	Effect
F1	RNG: deterministic (fixed-seed LCG) vs. entropy-seeded; regenerates the city
F2/F3/F4	Startup screen-quake on-off / weaker / stronger
F5	Variable vs. fixed simulation timestep
F6/F8	Sprite-reveal level / all-geometry↔all-sprites
F7	Particles on / off
F9/F10	Sound mute / music track
H	Perf + memory overlay; hover an entity to inspect it
+/-	Simulation time scale (slow-mo / fast-forward)
C/X/Z/G	Colliders / parallax / wireframe / grid overlays
P/0	Pause / single-step
F12	Save a PNG screenshot